DRIVER DROWSINESS DETECTION

DISSERTATION REPORT

Submitted by

ASIF ALI

2020-350-012

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

Under the supervision of

DR. MOHD. ABDUL AHAD



Department of Computer Science & Engineering School of Engineering Sciences & Technology

JAMIA HAMDARD

(Deemed to be University)

New Delhi-110062

2024

INDEX

S NO	TOPIC	PAGE NO
1.	CERTIFICATE	3
2.	DECLARATION	4
3.	ACKNOWLEGMENTS	5
4.	TITLE	6
5.	OBJECTIVE	7
6.	INTRODUCTION	8-12
7.	PROBLEM STATEMENT	13
8.	SRS in IEEE Format	14-16
9.	METHODOLOGY	17-19
11.	SNAPSHOTS	20-25
12.	RESULT IMAGES OF THE DETECTION	26-29
13.	CONCLUSION	30-32
14.	REFERENCES	33

OBJECTIVE

Object detection plays a crucial role in various applications within computer vision, including driver drowsiness detection. The aim of this project is to develop a robust driver drowsiness detection system using YOLOv8, an advanced object detection model. By leveraging YOLOv8's capabilities, we aim to accurately identify signs of drowsiness in drivers from images, videos, and real-time webcam feeds.

Key Components:

1. **Model Selection**: Utilize YOLOv8, a state-of-the-art object detection model, for driver drowsiness detection tasks. Compare its performance with other models to ensure optimal selection.
2. **Data Collection and Preparation**: Curate a diverse dataset containing images and videos depicting various levels of driver drowsiness. Augment the dataset to enhance model generalization and robustness.
3. **Model Training**: Train the YOLOv8 model on the collected dataset, optimizing hyperparameters and employing transfer learning techniques to expedite convergence and improve accuracy.
4. **Inference Implementation**: Implement inference pipelines for image, video, and real-time webcam feed analysis using the trained YOLOv8 model. Ensure real-time performance and accuracy for practical application scenarios.
5. **Evaluation Metrics**: Evaluate the performance of the YOLOv8-based drowsiness detection system using metrics such as precision, recall, F1-score, and mAP. Compare the results with existing methods to assess the efficacy and reliability of the proposed solution.

Expected Outcome:

By developing a robust driver drowsiness detection system based on YOLOv8, this project aims to contribute to enhanced road safety measures. The implementation of an efficient and accurate detection system can potentially mitigate the risks associated with drowsy driving, thereby reducing road accidents and improving overall transportation safety.

INTRODUCTION

In many applications, including those in autonomous driving, robotics, video surveillance, and augmented reality, object detection has become a crucial element. The YOLO (You Only Look Once) framework has distinguished itself among the numerous object detection algorithms for its exceptional balance of speed and precision, enabling the quick and accurate identification of objects in photos.

A well-known collection of object detection models called YOLO (You Only Look Once) is used in computer vision for real-time object recognition and classification.

YOLO, which was first created by Joseph Redmon, Ali Farhadi, and Santosh Divvala, aims to detect objects with high precision and speed in real-time. The model series falls under the category of single-stage object detection models, employing convolutional neural networks (CNNs) to analyse an entire image in a single forward pass.

YOLO's single-stage detection technique, which is intended to detect objects in real time and with high accuracy, is its major feature. YOLO processes the entire image in a single pass, making it faster and more effective than two-stage detection methods like R-CNN that first suggest regions of interest and then classify these regions.

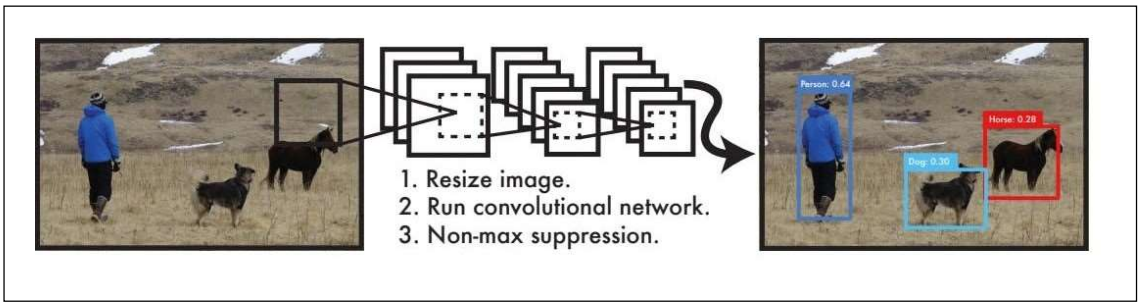
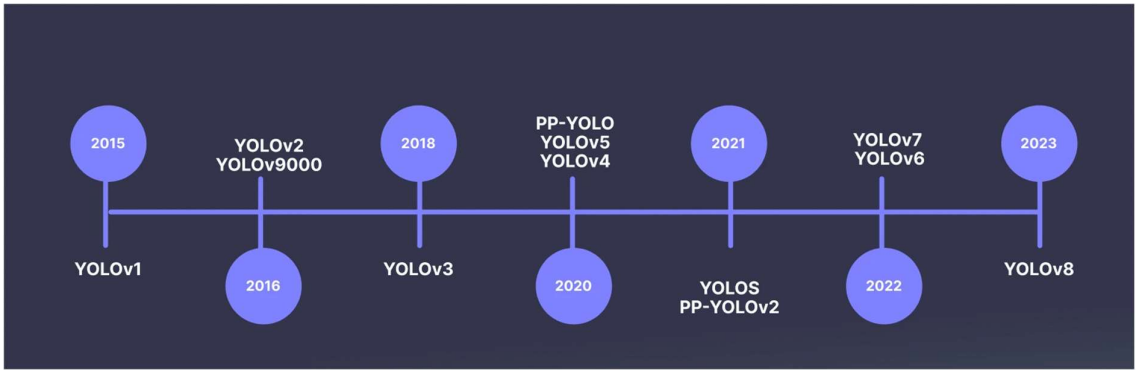


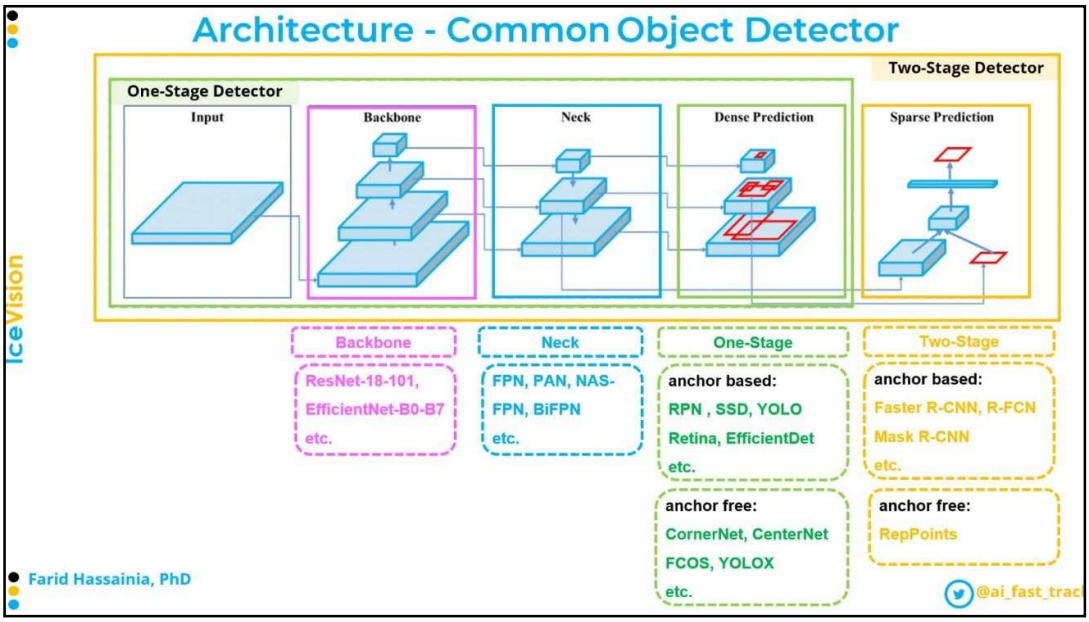
Fig.: YOLO Detection Process. Utilizing YOLO for image processing is a streamlined procedure. Our system conducts three steps: (1) resizing the input image to dimensions of448 × 448, (2) executing a singular convolutional network on the image, and (3) applying confidence-based thresholds to the resulting detections by the model.

- **Evolution Figure of YOLO**



- **High-level architecture for single-stage object detectors:**

The realm of object detection models encompasses two primary categories: **two-stageobject detectors** and **single-stage object detectors.**



To create dense predictions, single-stage object detector architectures (such as YOLO) are built of three components: a **Backbone**, a **Neck**, and a **Head**, as shown in the image below.

Model Backbone

The backbone is a trained network that extracts rich feature representations from images. This aids in lowering the image's spatial resolution while improving its feature (channel) resolution.

Model Neck

The feature pyramids are extracted using the model neck. This aids the model's generalization to objects of various sizes and scales.

Model Head

The model head is utilized to complete the final stage activities. It uses anchor boxes on feature maps to generate the final output, which includes **classes, objectness scores,** and **bounding boxes.**

YOLOv8 MODEL: -

YOLOv8 is the most recent version of the YOLO algorithm released in 2023, and it improves on earlier versions by integrating new modules such as spatial attention, feature fusion, and context aggregation.

These enhancements result in faster and more accurate object detection, elevating YOLOv8 to the forefront of object detection algorithms.

ARCHITECTUE: -

YOLOv8's architecture builds on previous versions of the YOLO (You Only Look Once) object detection models. YOLOv8 employs a fully convolutional neural network, which is separated into two major components: the backbone and the head.

The YOLOv8 architecture is based on a modified version of the CSPDarknet53 design. This architecture has 53 convolutional layers and uses a technique called cross-stage partial connections to increase information flow between the network's layers.

YOLOv8's head is made up of numerous convolutional layers followed by a succession of fully linked layers. These layers are in charge of predicting the bounding boxes, objectness scores, and class probabilities of objects recognized in images.

The usage of a self-attention mechanism in the network's brain is a major aspect of YOLOv8. This method enables the model to concentrate on different areas of the image and modify the importance of various attributes based on their relevance to the job.

Another key aspect of YOLOv8 is its capacity to recognize objects at several scales. To detect objects of various sizes and scales inside a picture, the model employs a feature pyramid network.

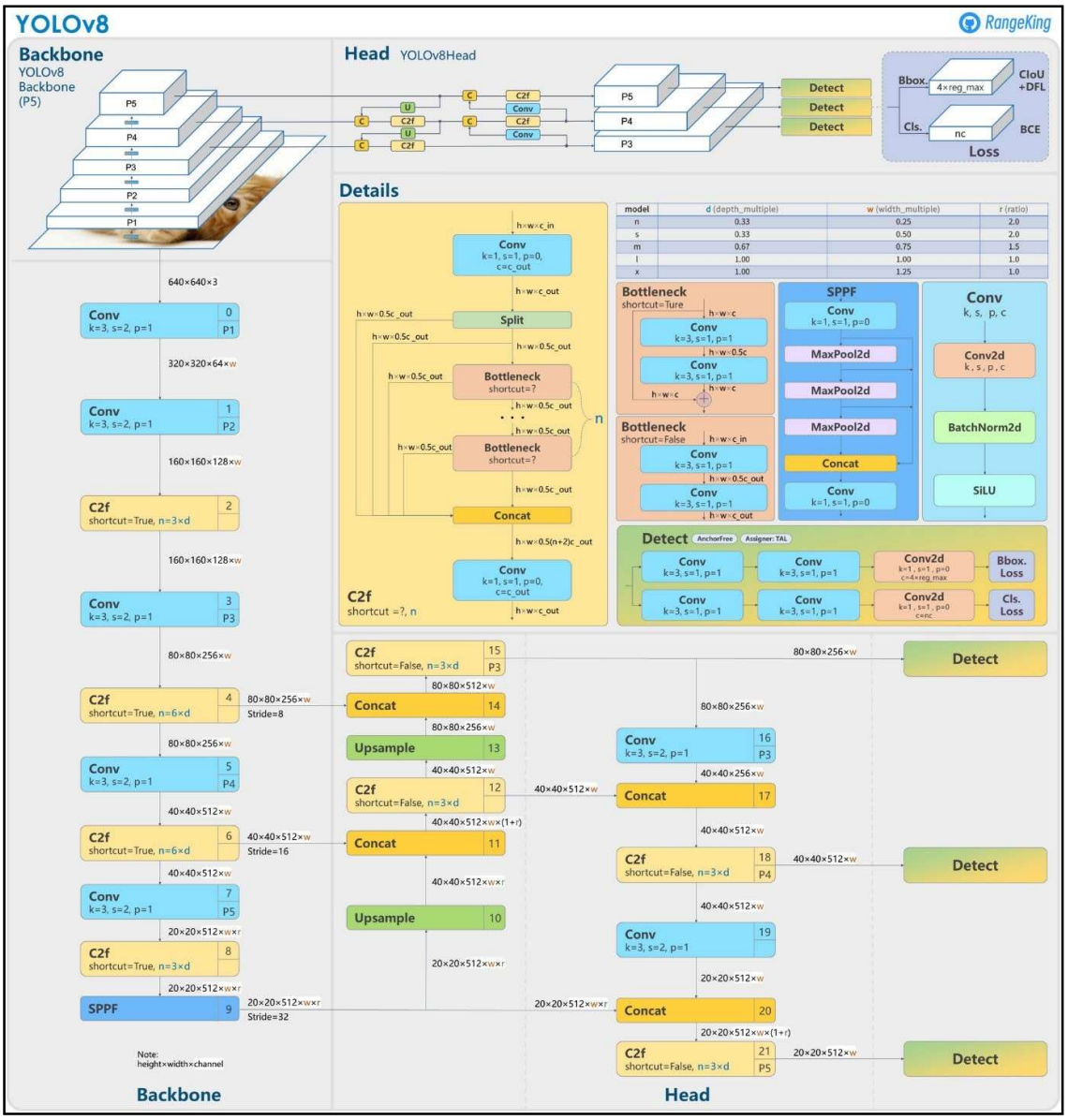


Fig:- YOLOv8 Architecture, visualization made by GitHub user RangeKing

PROBLEM STATEMENT

Driver drowsiness is a significant contributor to road accidents worldwide. Drowsy driving poses a serious threat to road safety, as it impairs the driver's ability to react promptly to changing road conditions, leading to accidents, injuries, and fatalities. Detecting signs of drowsiness in drivers is crucial for preventing accidents and ensuring safe transportation.

The primary challenge addressed in this project is the accurate and real-time detection of driver drowsiness from visual cues captured through images, videos, and live webcam feeds. Identifying subtle signs of drowsiness, such as drooping eyelids, yawning, and changes in facial expressions, presents a complex computer vision problem that requires robust detection algorithms.

Scope of the Problem

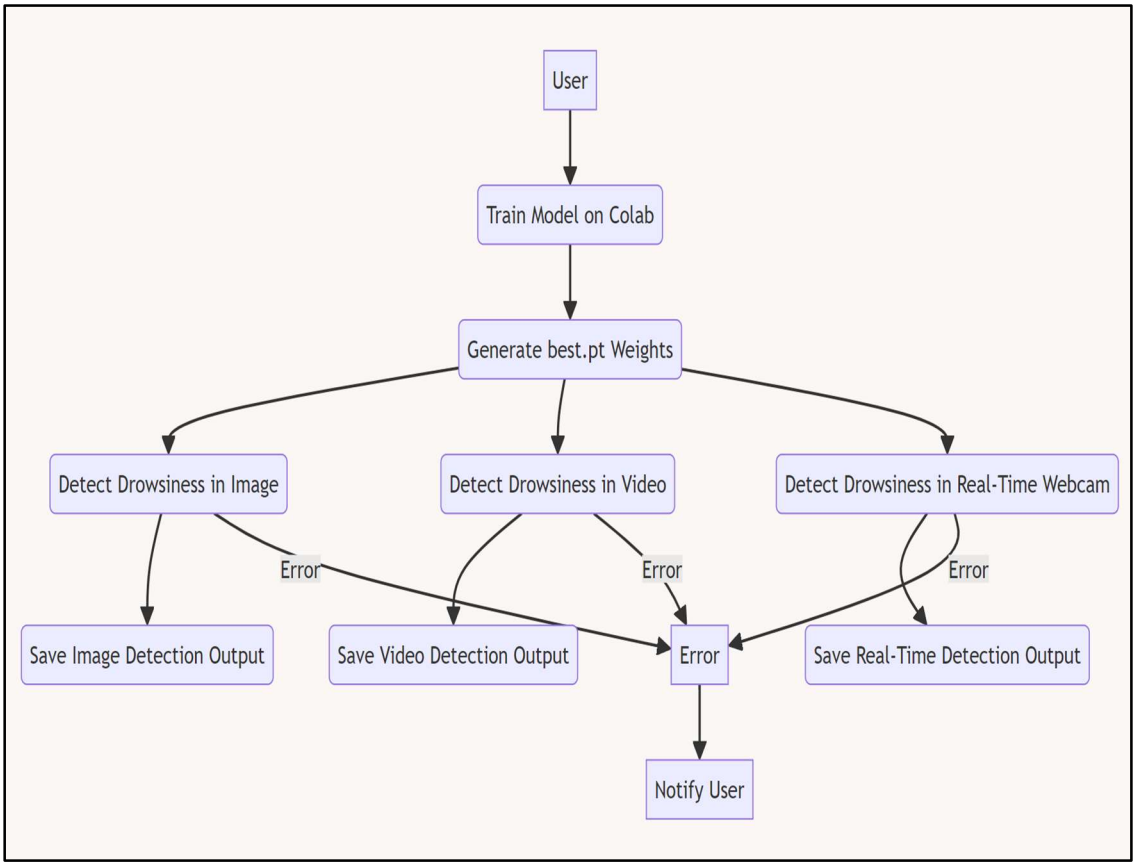
The scope of this project encompasses the development of a comprehensive driver drowsiness detection system capable of:

- Detecting various levels of drowsiness in drivers from static images, recorded videos, and live webcam feeds.
- Implementing efficient and accurate object detection algorithms to analyse visual cues indicative of drowsiness.
- Evaluating the performance of the detection system using standard metrics and comparing it with existing approaches.

YOLOv8, or "You Only Look Once" version 8, is a state-of-the-art object detection model known for its speed and accuracy. It offers real-time inference capabilities, making it suitable for applications requiring fast and reliable detection. By leveraging YOLOv8, we aim to address the challenges of driver drowsiness detection effectively. Its ability to detect multiple objects simultaneously with high accuracy makes it ideal for identifying subtle signs of drowsiness in drivers, such as closed eyes, yawning, and distracted behaviour.

SOFTWARE REQUIREMENT SPECIFICATION

1. Use Case Diagram



2. Use Case Specifications

Use Case Name	Pre-condition	Basic Path	Alternate Path	Post-condition	Exception Path
Train Model	Dataset is available and properly labelled.	User initiates the training process. The system trains the YOLOv8 model using the dataset.	N/A	The system generates the model weights (best.pt).	If the training process fails, the system notifies the user.
Detect Drowsiness in Image	The model is trained and an image is available.	User provides an image. The system performs drowsiness detection on the image.	N/A	The system provides the detection results.	If detection fails, the system notifies the user.
Detect Drowsiness in Video	The model is trained and a video is available.	User provides a video. The system performs drowsiness detection on the video.	N/A	The system provides the detection results.	If detection fails, the system notifies the user.
Detect Drowsiness in Real-Time	The model is trained and a webcam is available.	User initiates real-time detection. The system performs drowsiness detection using the webcam feed.	N/A	The system provides the detection results.	If detection fails, the system notifies the user.

3. Hardware requirements

Local Machine:

- Intel i5 10th gen
- 16GB RAM
- GTX 1650 TI 4GB VRAM

Google COLAB:

- Intel® Xeon® @ 2.30GHz 2 cores
- Nvidia K80 / T4 12GB Memory @ 0.82GHz 2 cores
- 12.7 GB RAM, 78.2 GB Storage

4. Software Requirements

Operating System:

- Windows 7 or later (64-bit)

GPU Drivers:

- A compatible NVIDIA GPU with CUDA support, accompanied by the most recent NVIDIA drivers for acceleration purposes.

Python Version:

- Python 3.9

Python Libraries:

- Ultralytics, TensorFlow, PyTorch, Keras

IDEs:

- Anaconda Jupyter Notebook, VS Code

METHODOLOGY

The methodology section of this report outlines the approach taken to develop a driver drowsiness detection system using the YOLOv8 model. The project involves several key steps, each of which plays a crucial role in the development and functioning of the system.

The steps include data preparation, where the necessary data is collected and organized for the model; model training, where the YOLOv8 model is trained on the prepared data; and detection, where the trained model is used to detect driver drowsiness in various scenarios.

Each of these steps is explained in detail in the following sections, providing a comprehensive overview of the process involved in the creation of this driver drowsiness detection system.

1. Data Preparation

The data consists of images and videos that are used to train the YOLOv8 model. The data is stored in a specific directory, and the path to this directory is provided to the model during the training phase.

The dataset contains images labeled with various classes representing different states of a driver, such as **“awake”, “calling”, “chatting”, “closed_eye”, “drink”, “drowsy”, “eating”, “no_yawn”, “open_eye”, “smoking”, and “yawn”**. These labels are used by the model to learn to detect these states in new images.

The **data.yaml** file is crucial for training the YOLOv8 model. It contains the paths to the training and validation datasets, as well as the list of class names. The model uses this file to know where to find the data and what classes to predict.

2. Model Training

The model training is performed using the ultralytics package, which provides a Python implementation of the YOLOv8 model. The model is initialized with the pre-trained weights from the file yolov8m.pt. The model is then trained on the prepared data for 200 epochs. The training process is performed on a GPU if one is available, otherwise, it falls back to using the CPU.

```
model = YOLO("yolov8m.pt").to(device)
model.train(data="C:/Users/ASIF ALI/1. FINAL SEM PROJECT/Drive Drowsiness
Detection.v3i.yolov8/data.yaml" , epochs = 200)
```

After the training, the model saves the best weights to a file named **best.pt**. These weights represent the state of the model that achieved the best performance on the validation data during training.

3. Detection

The detection process involves passing the input data (image, video frame, or webcam frame) through the model, which returns a list of detected objects. Each object is represented by a bounding box and a class label.

The model used for detection is initialized with the best.pt weights. These weights represent the state of the model that achieved the best performance on the validation data during training.

```
model = YOLO("C:/Users/ASIF ALI/1. FINAL SEM PROJECT/Drive Drowsiness
Detection.v3i.yolov8/runs/detect/train/weights/best.pt")
```

4. Image Detection

For image detection, an image is loaded and passed through the model. The model returns a list of detected objects, each with a bounding box and a class label. The bounding boxes and labels are then drawn on the image. The processed image is saved to a specified location.

```
img = cv2.imread(r'C:\Users\ASIF ALI\1. FINAL SEM PROJECT\DATASET IMG\d12.jpg')
results = model(img, stream=True)
cv2.imwrite('C:/Users/ASIF ALI/1. FINAL SEM PROJECT/Outputs/Images/IMGoutput.jpg', img)
```

5. Video Detection

For video detection, a video file is opened and each frame of the video is passed through the model. The model returns a list of detected objects for each frame, and the bounding boxes and labels are drawn on the frame. The processed frames are then written to an output video file.

```
cap = cv2.VideoCapture(video_path)
for frame in video_detection(cap):
    out.write(frame)
out.release()
```

6. Real-Time Detection

For real-time detection, a webcam is used to capture video frames in real-time. Each frame is passed through the model, and the detected objects are drawn on the frame. The processed frames are displayed in a window and also written to an output video file.

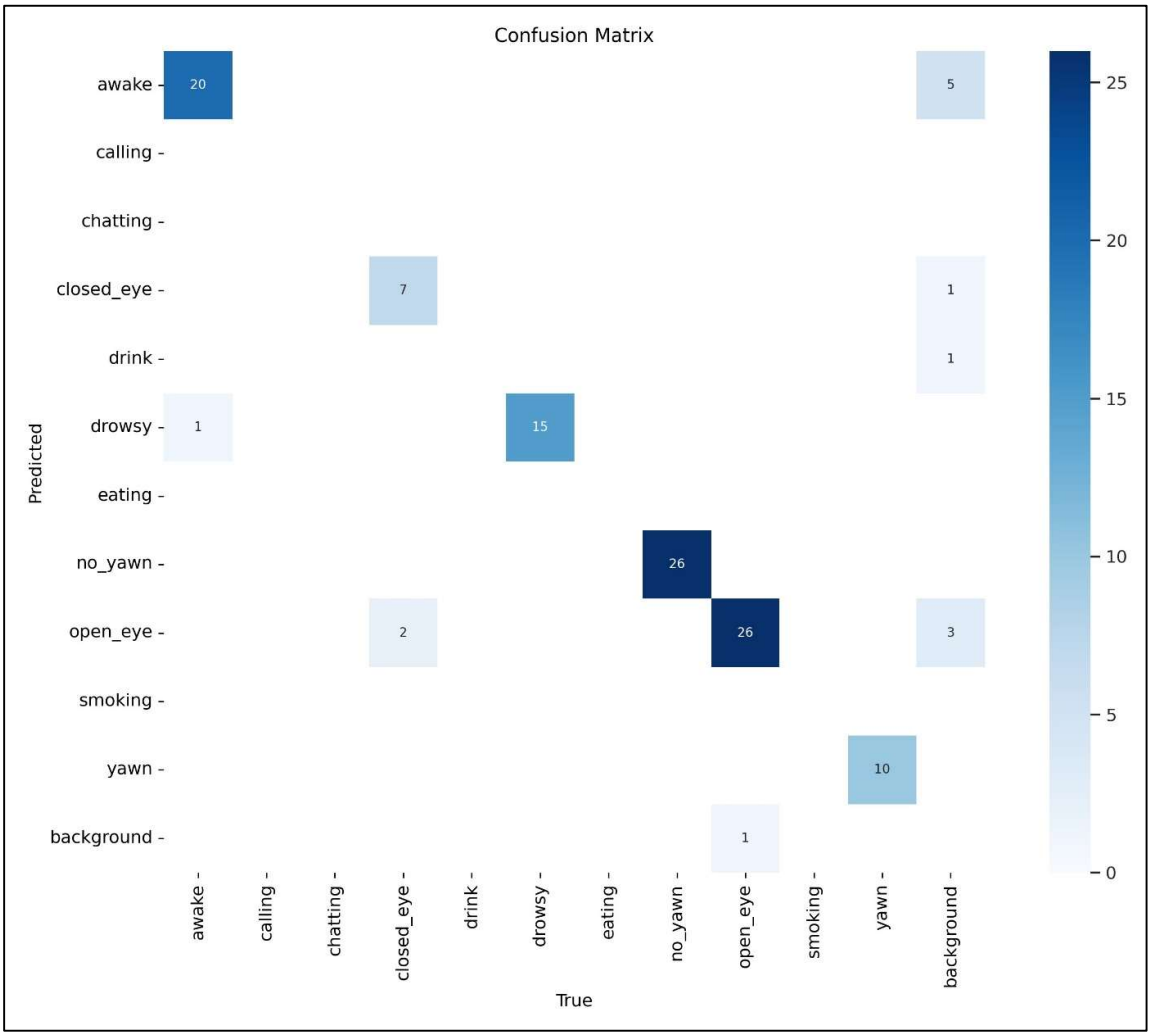
```
cap = cv2.VideoCapture(0)
for frame in video_detection(cap):
    out.write(frame)
out.release()
```

In all three cases, the best.pt weights are used to initialize the model for detection. The processed images or video frames are then saved to the specified locations.

SNAPSHOTS OF RESULTS

Confusion Matrix:

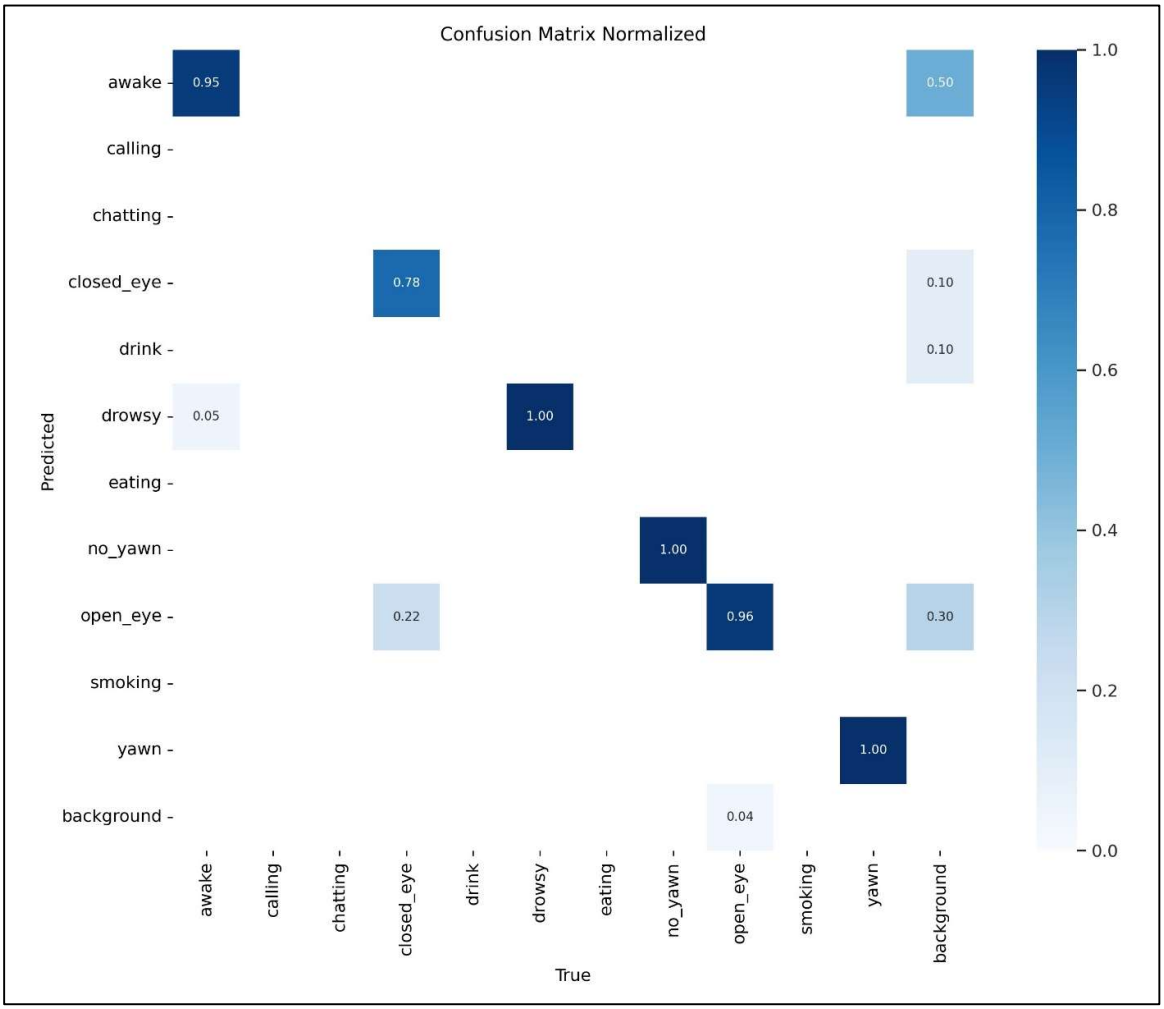
The confusion matrix visualizes YOLOv8 model's performance. The diagonal cells show correct predictions, with "no yawn" being the most accurate. Off-diagonal cells indicate misclassifications, where the model predicted one class but the actual class was different.



Confusion Matrix Normalized:

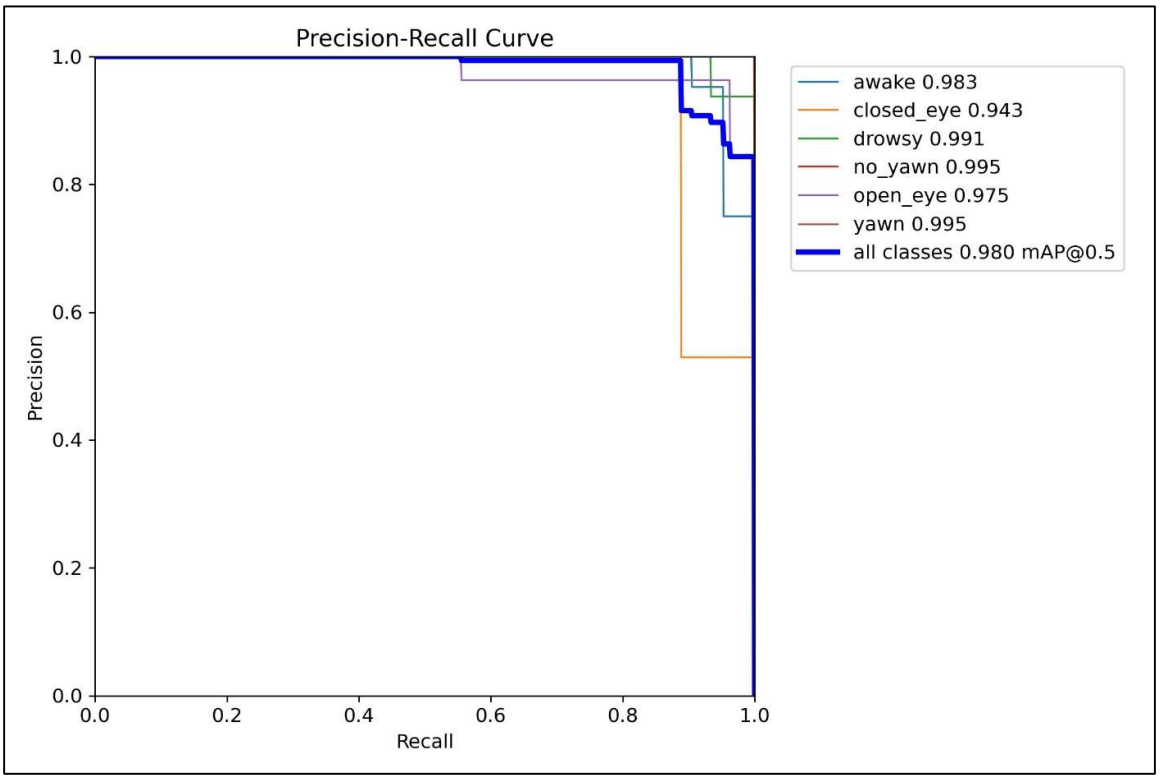
Normalized confusion matrix for YOLOv8 model shows the proportion of predictions for each combination of true and predicted categories. **The diagonal cells represent correct predictions, with "awake", "no yawn", and "yawn" having high values like 0.95 and 1.00,** indicating high accuracy.

Off-diagonal cells indicate misclassifications, where the model predicted one class but the actual class was different. Lighter squares such as 0.22 for "open_eye" indicate lower accuracy. This matrix is a powerful tool for understanding the performance of model across different classes.



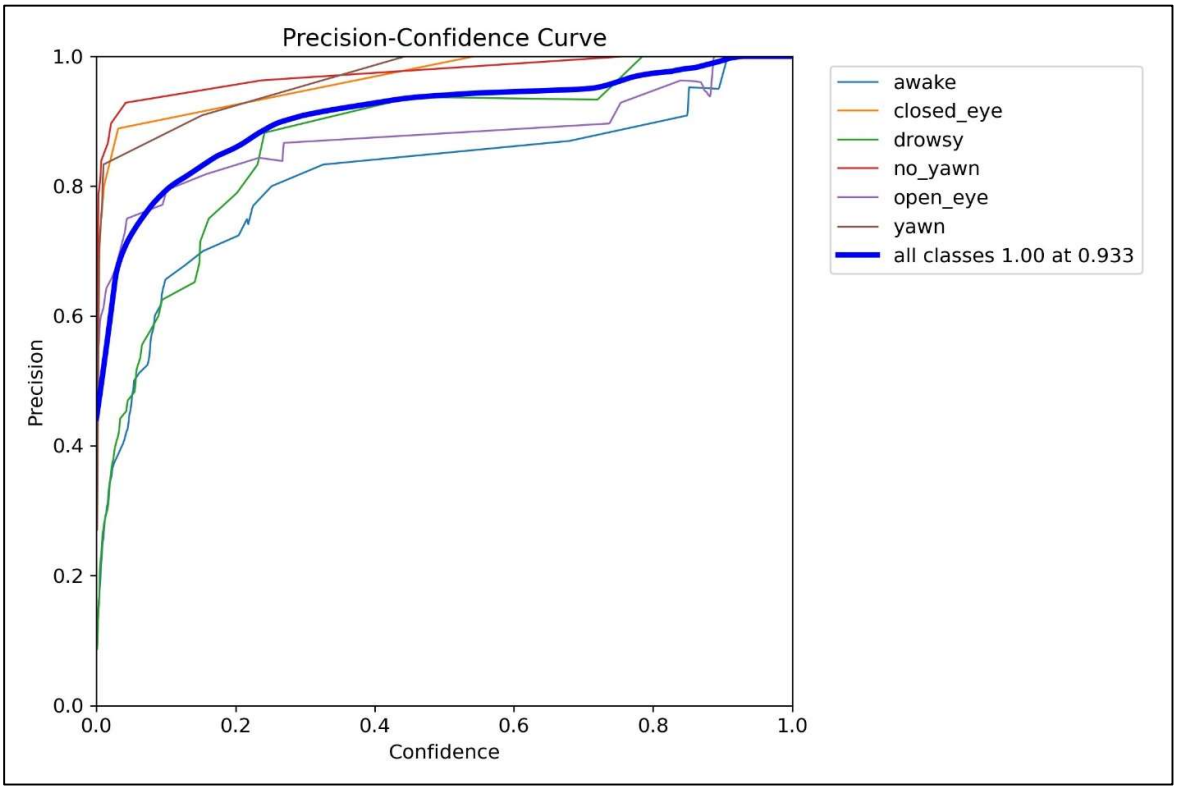
Precision-Recall Curve (PR-curve):

Precision-Recall Curve graph for YOLOv8 model shows the relationship between precision and recall for different classes like “awake”, “closed eye”, “drowsy”, “no yawn”, “open eye”, and “yawn”. **The mean average precision (mAP) for all classes at an Intersection over Union (IoU) of 0.5 is 0.980**, indicating a high level of accuracy in model’s predictions. IoU, or Intersection over Union, quantifies the overlap between two bounding boxes.. Each class also has a high precision value, suggesting that model performs well across all classes.



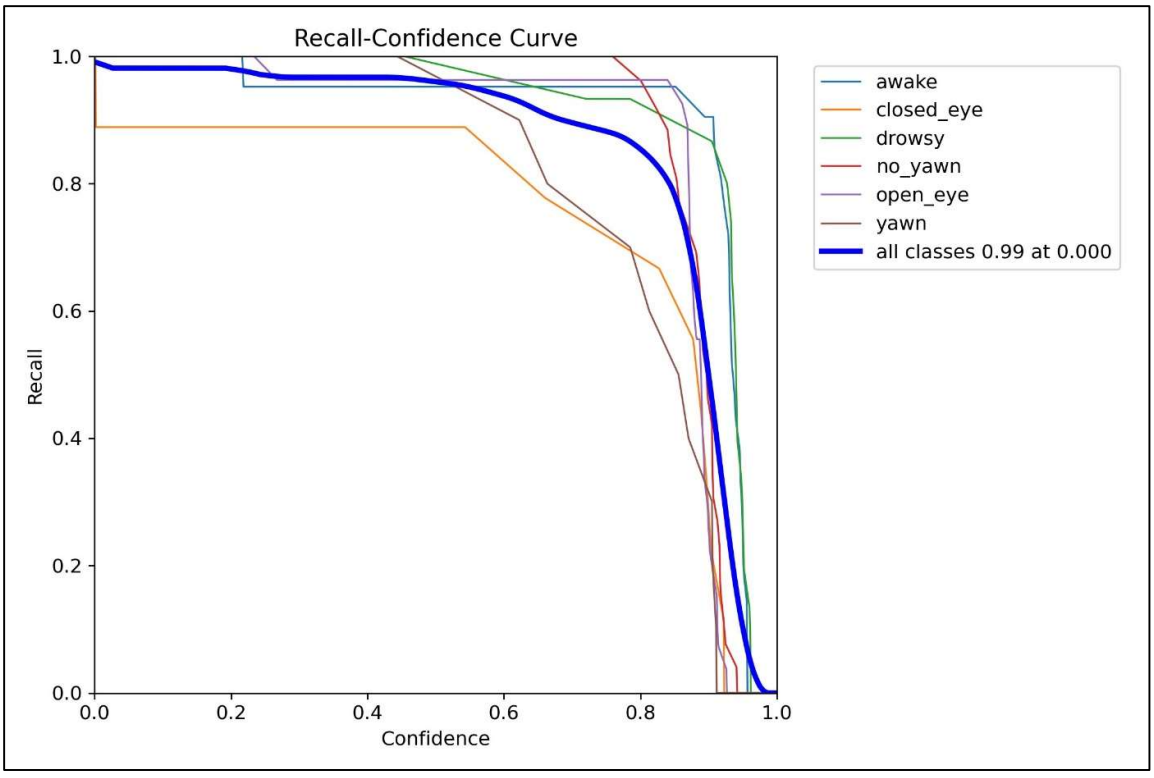
Precision-Curve (P-curve):

Precision-Confidence Curve graph for YOLOv8 model. It shows the relationship between precision and confidence for different classes like “awake”, “closed eye”, “drowsy”, “no yawn”, “open eye”, and “yawn”. **The graph indicates that for a confidence of 0.933, the precision for all classes is 1.00**, suggesting a high level of accuracy in model’s predictions. Confidence here refers to the model’s certainty in its predictions. Each class also has a high precision value, indicating that model performs well across all classes.



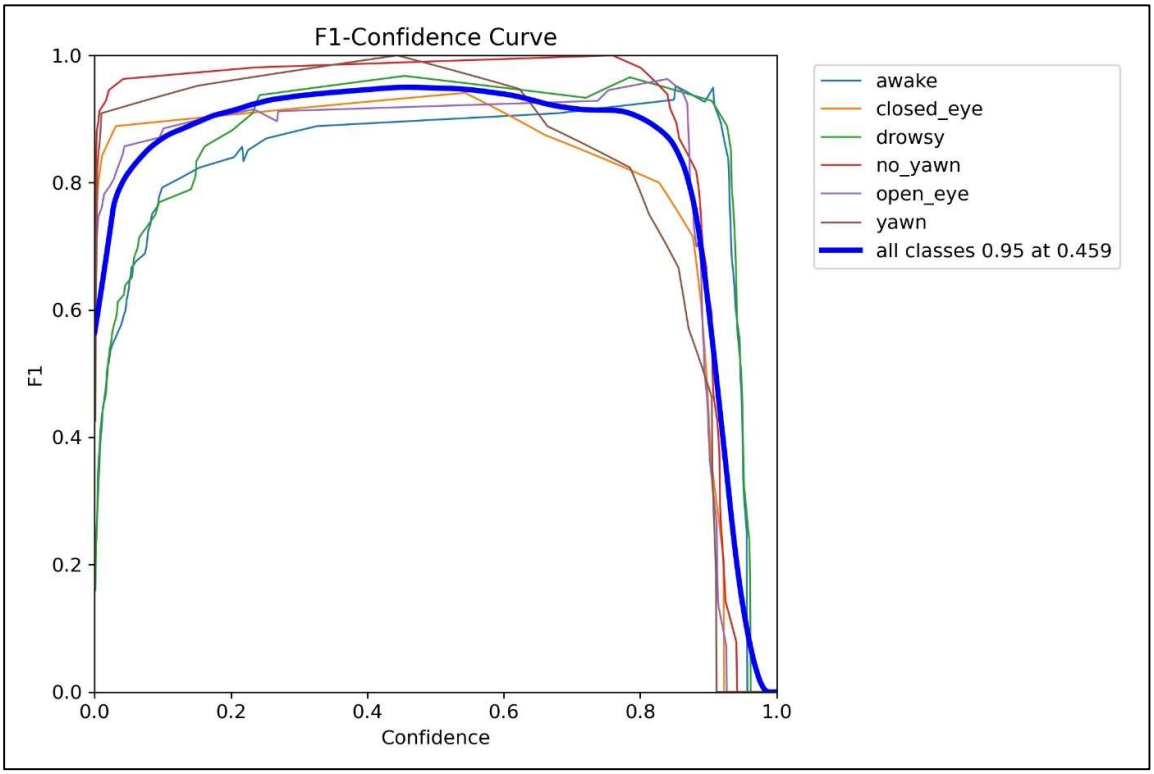
Recall-Curve (R-curve):

Recall-Confidence Curve graph for YOLOv8 model. It shows the relationship between recall and confidence for different classes like “awake”, “closed eye”, “drowsy”, “no yawn”, “open eye”, and “yawn”. **The graph indicates that for a confidence of 0.000, the recall for all classes is 0.99**, suggesting a high level of accuracy in model’s predictions. Confidence here refers to the model’s certainty in its predictions. Each class also has a high recall value, indicating that model performs well across all classes.



F1-Curve:

F1-Confidence Curve graph for YOLOv8 model shows the relationship between F1 score and confidence for different classes like “awake”, “closed eye”, “drowsy”, “no yawn”, “open eye”, and “yawn”. **The graph indicates that for a confidence of 0.459, the F1 score for all classes is 0.95,** suggesting a high level of accuracy in your model’s predictions. Confidence here refers to the model’s certainty in its predictions. Each class also has a high F1 score value, indicating that model performs well across all classes.



RESULT IMAGES OF THE DETECTION

1. Image Detection:

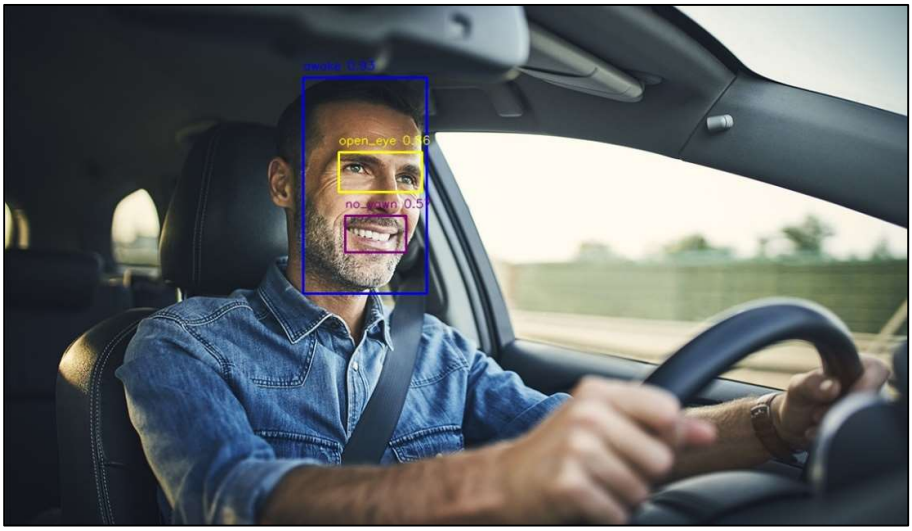


FIG: Img-1

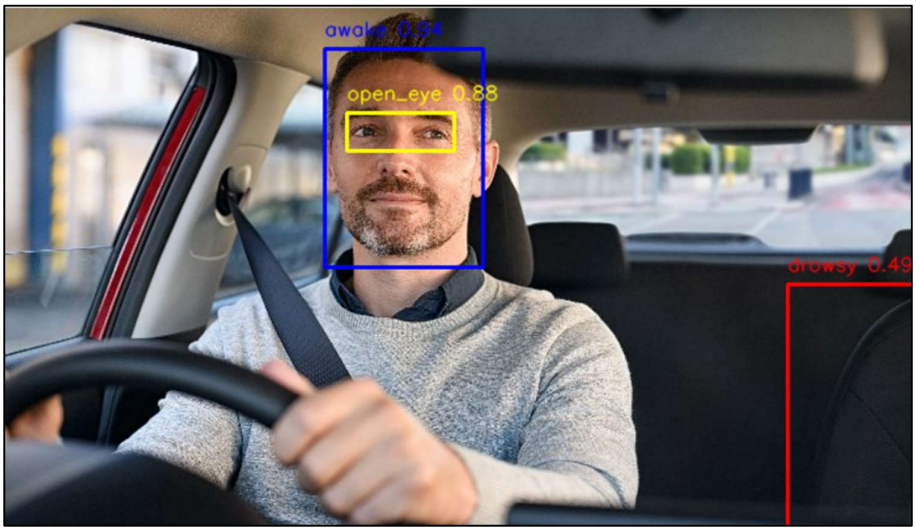


FIG: Img-2

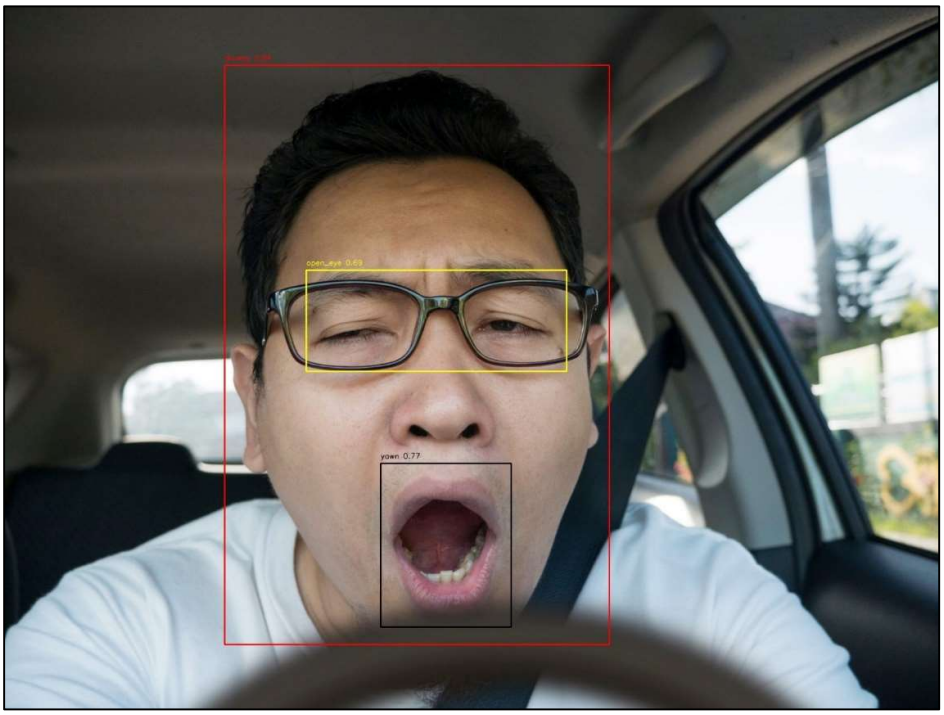


FIG: Img-3



FIG: Img-4

2. Video Detection:

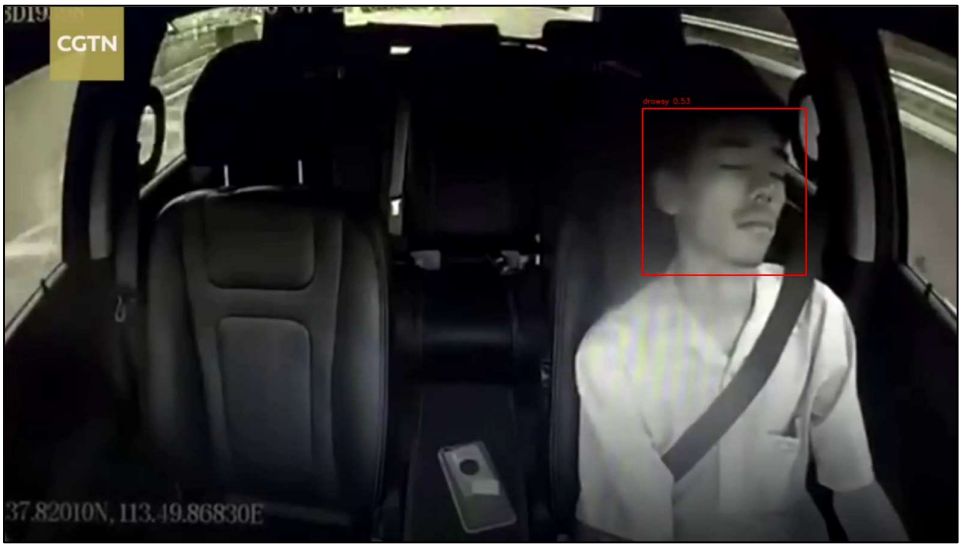


FIG: Img-5



FIG: Img-6

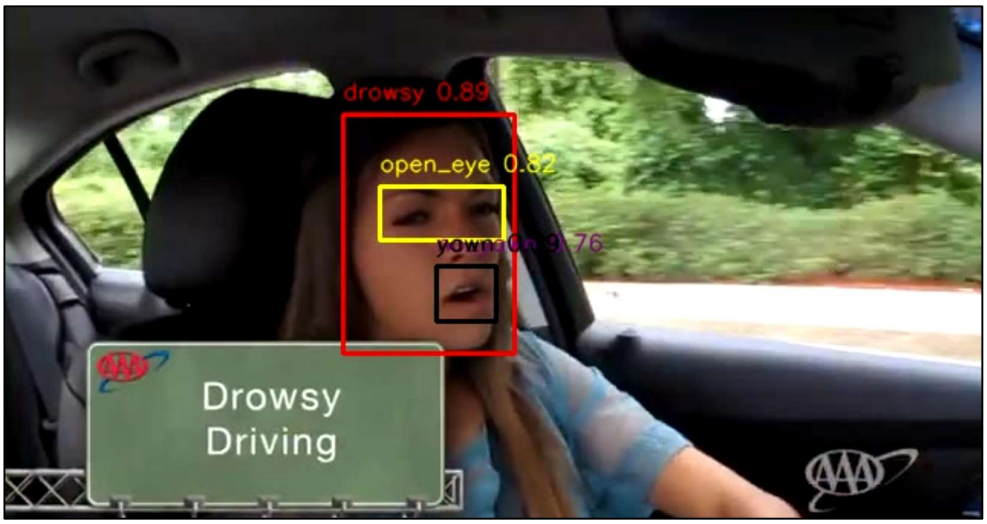


FIG: Img-7

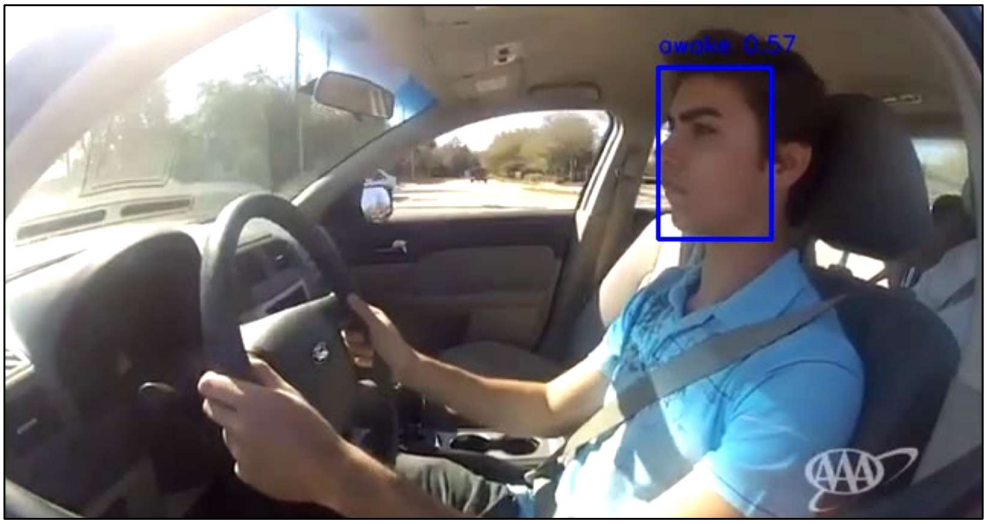


FIG: Img-8

CONCLUSION

The objective of this project was to develop a robust and efficient model for driver drowsiness detection using YOLOv8. The model was trained to classify different states such as “awake”, “closed eye”, “drowsy”, “no yawn”, “open eye”, and “yawn”.

Table for the result of detected objects:

Image S No.	Object Detected	Confidence Score
Img-1	3 (awake, open_eye, no_yawn)	0.93, 0.86, 0.57
Img-2	2+1 Error (awake, open_eye, drowsy)	0.94, 0.88, 0.49
Img-3	3 (drowsy, open_eye, yawn)	0.84, 0.69, 0.77
Img-4	1 (drowsy)	0.47
Img-5	1 (drowsy)	0.53
Img-6	1 (drowsy)	0.32
Img-7	3 (drowsy, open_eye, yawn)	0.89, 0.82, 0.90
Img-8	1 (awake)	0.57

In addition to these curves, a detailed table of results was also generated for eight different images. The table provided the detected objects and their respective confidence scores for each image. For instance, in the first image, three objects were detected: “awake”, “open eye”, and “no yawn” with confidence scores of 0.93, 0.86, and 0.57 respectively. In some images like Img-4 and Img-5, only one state “drowsy” was detected with lower confidence scores.

However, it's important to note that there was an error in the second row of the table labeled as "2+1 Error." This indicate a misclassification or a failure in detection during the processing of the second image. Such errors provide valuable insights into the areas where the model might need improvement.

The performance of the model was evaluated using various metrics such as precision, recall, and mean average precision (mAP). These metrics offer a thorough assessment of the model's performance across all classes. The results were visualized through different curves including the Recall-Confidence Curve, Precision-Recall Curve, and Precision-Confidence Curve.

Metric	Value	Confidence
Recall	0.99	0.000
Mean Average Precision (mAP)	0.980	N/A
Precision	1.00	0.933
F1 Score	0.95	0.459

The **Recall-Confidence Curve** graph indicated a high level of accuracy in the model's predictions, with a recall of 0.99 for all classes at a confidence of 0.000. This indicates that the model performs well across all classes, accurately identifying the correct class for a high proportion of instances.

The **Precision-Recall Curve** graph showed a mean average precision (mAP) for all classes at an Intersection over Union (IoU) of 0.5 of 0.980. This high mAP score indicates a high level of accuracy in the model's predictions, with a good balance between precision and recall.

The **Precision-Confidence Curve** graph showed that for a confidence of 0.933, the precision for all classes is 1.00, suggesting a high level of accuracy in the model's predictions. This indicates that when the model is highly confident in its predictions, it is usually correct.

The **F1-Confidence Curve** graph showed that for a confidence of 0.459, the F1 score for all classes is 0.95, indicating a high level of accuracy in model's predictions. The F1 score, a metric for evaluating test accuracy, accounts for both precision and recall. The high F1 score

indicates that model performs well across all classes, balancing both precision (how many selected items are relevant) and recall (how many relevant items are selected).

The **Confusion Matrix** provided further evidence of the model's performance. The diagonal cells, which represent correct predictions, showed "no yawn" as the most accurate prediction. The off-diagonal cells, which represent misclassifications, were minimal, indicating that the model performs well across all classes.

In conclusion, the **YOLOv8 model demonstrated high performance in detecting driver drowsiness.** The high scores across various metrics and the detailed analysis of the curves suggest that the model is robust and reliable. The model's ability to accurately classify different states could have significant implications in various applications such as driver drowsiness detection systems, which could potentially save lives by alerting drivers when they are drowsy.

Future work could explore further optimization techniques to improve the model's performance. Additionally, the model could be applied to other similar tasks to further validate its robustness and reliability. The success of this project serves as a testament to the potential of deep learning models in solving complex classification problems.